

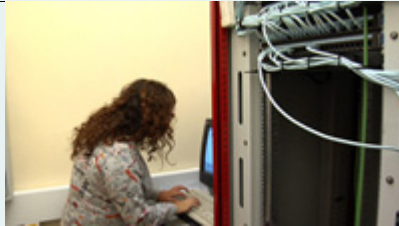
Técnicas de programación segura



Caso práctico

Juan esta terminado de realizar una aplicación.

María: Hola Juan. Ya he visto que estás terminado la aplicación para la empresa. Recientemente hemos tenido bastantes problemas con la seguridad en las aplicaciones desarrolladas porque cada vez es más frecuente sufrir ataques informáticos.



[Ministerio de Educación](#) (Uso educativo [nc](#))

Juan: ¿Y qué tengo que tener en cuenta? Pensaba que la seguridad informática era una función de los administradores de la red y de los servidores. Ya sabes, cortafuegos, gestión de usuarios,...

María. Una parte muy importante de la seguridad de la empresa reside en la red y en los servidores. Pero ten en cuenta, que cuando el usuario puede ejecutar la aplicación, la seguridad del sistema queda en la aplicación. Tenemos que tener mucho cuidado en la seguridad de las aplicaciones que desarrollamos. Mira, a continuación, vamos a ver una introducción a la seguridad informática haciendo hincapié en las aplicaciones y luego entraremos en más detalle, en la programación de aplicaciones seguras.



[Ministerio de Educación, Formación Profesional y Deportes](#) (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

[Aviso Legal](#)

1.- Introducción a la seguridad Informática



Caso práctico

María: Mira Juan. Ahora vamos a ver los conceptos más importantes sobre la seguridad informática, para saber cuándo un sistema o aplicación es seguro. Después, nos centraremos en la seguridad de las aplicaciones, para saber como programador qué recomendaciones tenemos que tener en cuenta.



Q [Ministerio de Educación](#) (Uso educativo [nc](#))

Juan: Ok, estupendo. Estoy deseando empezar...

Según la Real Academia Española, seguridad es "la cualidad de seguro, es decir, de estar libre y exento de todo daño, peligro o riesgo". En informática, como en tantas facetas de la vida, la seguridad entendida según la definición anterior es prácticamente imposible de conseguir, por lo que se ha relajado acercándose más al concepto de fiabilidad; se entiende un sistema seguro como aquel que se comporta como se espera de él.



Q [jhnri4](#) (OpenClipArt)

De los sistemas informáticos, ya sean sistemas operativos, servicios o aplicaciones, se dice que son seguros si cumplen las siguientes características:

- **Confidencialidad.** Requiere que la información sea accesible únicamente para las entidades autorizadas.
- **Integridad** . Requiere que la información sólo pueda ser modificada por las entidades autorizadas. La modificación incluye escritura, cambio, borrado, creación y reenvío de los mensajes transmitidos.
- **No repudio** . Ofrece protección a un usuario frente a otro que niegue posteriormente que se realizó cierta comunicación. El no repudio de origen protege al receptor de que el emisor niegue haber enviado el mensaje, mientras que el no repudio de recepción protege al emisor de que el receptor niegue haber recibido el mensaje. Las firmas digitales constituyen el mecanismo más empleado para este fin.
- **Disponibilidad.** Requiere que los recursos del sistema informático estén disponibles para las entidades autorizadas cuando los necesiten.

El tratamiento de los posibles incidentes de seguridad exige que toda organización tenga definida una política de seguridad, cuya función sea establecer las responsabilidades y reglas necesarias para evitar amenazas y minimizar los efectos de éstas. Lo primero que se debe realizar es identificar qué queremos proteger.

Desde el punto de vista informático, existen tres tipos de elementos que pueden sufrir amenazas:

- Hardware
- Software
- Datos

El más sensible, y en el que se basa casi toda la literatura sobre seguridad, son los datos, ya que es el único elemento que depende exclusivamente de la organización. Es decir, tanto el hardware como el software, si en la peor de las situaciones se pierden, siempre se pueden adquirir y/o instalarlos; pero los datos pertenecen a la organización y nadie puede, en el caso de pérdida, proporcionarlos. Sin embargo, la seguridad se debe entender a todos los niveles y aplicarla a todos los elementos.



Autoevaluación

¿Qué característica no es propia de un sistema seguro?

- ☐ Integridad.
- ☐ Disponibilidad.
- ☐ Rapidez.
- ☐ No repudio.

No es correcto, repasa los conocimientos de este apartado.

No, repasa los conocimientos del apartado.

Correcto.

Incorrecto, repasa el apartado.

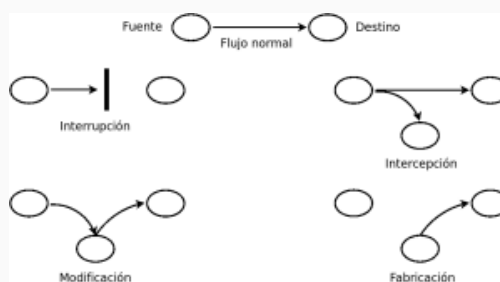
Solución

1. Incorrecto (#answer-5_63)
2. Incorrecto (#answer-5_78)
3. Opción correcta (#answer-5_81)

4. Incorrecto (#answer-5_84)

1.1.- Amenazas de seguridad

Se entiende por amenaza una condición del entorno del sistema de información (por ejemplo: persona, máquina) que, dada una oportunidad, podría dar lugar a que se produjese una violación de la seguridad (confidencialidad, integridad, disponibilidad o uso legítimo).



Q Ministerio de Educación (Uso educativo no)

Las amenazas de seguridad pueden caracterizarse modelando el sistema como un flujo de información; desde una fuente, como por ejemplo un fichero o un equipo, a un destino, como por ejemplo otro fichero o un usuario.

Tal y como puede ver en la figura las cuatro categorías generales de amenazas o ataques son las siguientes:

- **Interrupción.** Un recurso del sistema es destruido o deja de estar disponible. Éste es un ataque contra la disponibilidad. Un ejemplo de ataque es la destrucción de un elemento hardware, como un disco duro, cortar una línea de comunicación o deshabilitar el sistema de gestión de ficheros.
- **Intercepción** . Una entidad no autorizada consigue acceso a un recurso. Éste es un ataque contra la confidencialidad. La entidad no autorizada puede ser una persona, un programa o un ordenador. Un ejemplo de este ataque es escuchar una línea para registrar los datos que circulen por la red y la copia ilícita de ficheros o programas (intercepción de datos), o bien la lectura de las cabeceras de paquetes para desvelar la identidad de uno o más de los usuarios implicados en la comunicación observada ilegalmente (intercepción de identidad).
- **Modificación.** Una entidad no autorizada no sólo consigue acceder a un recurso, sino que es capaz de manipularlo. Éste es un ataque contra la integridad. Un ejemplo de este ataque es alterar un programa para que funcione de forma diferente, modificar el contenido de un fichero o de un mensaje transferido por red.
- **Fabricación** . Un ataque contra la autenticidad tiene lugar cuando una entidad no autorizada inserta objetos falsificados en el sistema. Un ejemplo de este ataque es la inserción de mensajes espurios (mensajes basura) en una red o añadir registros a un fichero.



Autoevaluación

¿En qué categoría de amenaza se clasifica un programa para escuchar el tráfico de la red?

- ☐ Interrupción.
- ☐ Intercepción.

- ☐ Modificación.
- ☐ Fabricación.

No, repasa los conocimientos del apartado.

Correcto.

Incorrecto.

No es correcto, repasa de nuevo.

Solución

1. Incorrecto (#answer-7_63)
2. Opción correcta (#answer-7_89)
3. Incorrecto (#answer-7_92)
4. Incorrecto (#answer-7_95)

1.2.- Ataques

Los ataques se pueden clasificar de forma útil en términos de ataques pasivos y ataques activos.

- Ataques pasivos. En los ataques pasivos, el atacante no altera la comunicación, sino que únicamente la escucha o monitoriza para obtener información de lo que está siendo transmitido. El principal uso que se suele realizar de los ataques pasivos es para ver el tráfico de un equipo, obtener las contraseñas de red, etc.

Los ataques pasivos son muy difíciles de detectar, ya que no provocan ninguna alteración de los datos. Sin embargo, es posible evitar su éxito mediante el cifrado de la información.

- Ataques activos. Estos ataques implican algún tipo de modificación del flujo de datos transmitido o la creación de un falso flujo de datos. Algunas formas puede ser: suplantación de identidad (el intruso se hace pasar por un usuario legítimo), reactuación (se reenvían mensajes legítimos), modificación de mensajes, denegación de servicio (se satura un servicio para que no pueda ser utilizado correctamente), etc.



Q [dias](#) (CC0)



Para saber más

Si deseas más información sobre los diferentes tipos de ataques informáticos puedes consultar el siguiente enlace:

[Tabla de ataques informáticos](#)

1.3.- Vulnerabilidades en el software

Debido al auge de Internet, las vulnerabilidades de software son uno de los mayores problemas de la informática actualmente. Las aplicaciones actuales cada vez tratan con más datos (personales, bancarios, etc.) y por eso, cada vez es más necesario que las aplicaciones sean seguras.



Q AJ (CC0)

Una vulnerabilidad de software puede definirse como un fallo o hueco de seguridad detectado en algún programa o sistema informático que puede ser utilizado para entrar en los sistemas de forma no autorizada.

De una forma sencilla, se trata de un fallo de diseño de algún programa, que puede tener instalado en su equipo, y que permite a los atacantes realizar una acción maliciosa. Si nos centramos en una aplicación, es posible que ese fallo de seguridad permita a un usuario que acceda o manipule a una información a la que no esté autorizado.

Pero el gran problema de los agujeros de seguridad reside en la forma en que son detectados. Cuando una aplicación tiene una cierta envergadura, se encarga el trabajo de detectar fallos de seguridad a usuarios externos para que utilicen la aplicación normalmente y/o a expertos de seguridad para que analicen a fondo la seguridad de la aplicación.

A pesar de que antes de lanzar una aplicación al mercado se intentan descubrir y solucionar todos sus fallos de seguridad, en muchas ocasiones se descubren después y por lo tanto, es necesario informar a los usuarios y permitir que se actualicen a la última versión que se encuentre libre de fallos.

Para poder garantizar la seguridad de una aplicación en Java nos vamos a centrar en dos pilares:

- **Seguridad interna de aplicación.** A la hora de realizar la aplicación se debe programar de una forma robusta para que comporte tal y como esperamos de ella. Algunas de las técnicas que podemos implementar, y que veremos en el segundo apartado, es la gestión de excepciones, validaciones de entradas de datos y la utilización de los ficheros de registro.
- **Políticas de acceso.** Una vez que tenemos una aplicación "segura" es importante definir las políticas de acceso para determinar las acciones que puede realizar la aplicación en nuestro equipo. Por ejemplo, podemos indicar que la aplicación pueda leer los ficheros de una determinada carpeta, enviar datos a través de Internet a un determinado equipo, etc. De esta forma, aunque un usuario malicioso utilice la aplicación de forma incorrecta, se limita el impacto de su ataque. Así, si hemos indicado que sólo puede leer los ficheros de la carpeta `c:/datos`, el atacante nunca podrá modificar esos datos o leer los ficheros de otro directorio.



¿Qué permiten las políticas de acceso?:

- ☐ Indicar los recursos a los que puede acceder una determinada aplicación.
- ☐ Gestionar los usuarios del sistema.
- ☐ Permite el acceso del usuario al sistema de forma correcta.

Correcto.

No es correcto, repasa los conocimientos del apartado.

Incorrecto.

Solución

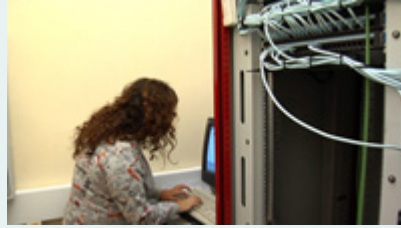
1. Incorrecto (#answer-11_63)
2. Incorrecto (#answer-11_102)
3. Opción correcta (#answer-11_105)

2.- Programación segura



Caso práctico

Juan: Hola María, ya tengo claro los conceptos básicos de seguridad y estoy deseando poder empezar a utilizarlos en mis programas. ¿Puedes orientarme?



María: Claro que sí Juan. Mira, [Q Ministerio de Educación \(Uso educativo nc\)](#) para tener aplicaciones en Java seguras, primero tenemos que tener en cuenta unas medidas básicas de seguridad que nos van a permitir mejorar la seguridad de nuestras aplicaciones. Por ejemplo, las excepciones permiten que el código se ejecute siempre correctamente, la validación de datos evita ataques y garantiza la consistencia de la información... Vamos a ir viendo todo esto para que puedas utilizarlo en tus programas.

Juan: ¡Estupendo!

2.1.- Excepciones

Una excepción es un evento que ocurre durante de la ejecución de un programa e interrumpe el flujo normal de las instrucciones.

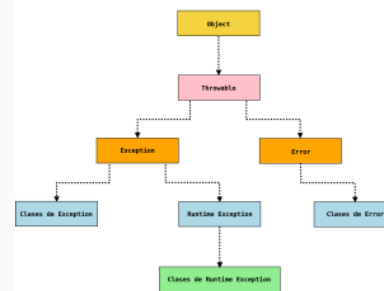
Por ejemplo, si desea crear una aplicación que lea un archivo y lo envíe por la red, algunas de las posibles excepciones son: que el dispositivo donde se almacena el fichero no esté disponible, que no tenga permisos de lectura sobre el fichero, que el fichero no exista, que la red no esté disponible, etc.

Es importante gestionar las posibles excepciones que puedan ocurrir en el programa para evitar cualquier tipo de fallos en su ejecución.

En Java existen varios tipos fundamentales de excepciones:

- Error. [Excepciones](#)  que indican problemas muy graves, que suelen ser no recuperables y no deben casi nunca ser capturadas.
- Exception. Excepciones no definitivas, pero que se detectan fuera del tiempo de ejecución.
- RuntimeException. Excepciones que se dan durante la ejecución del programa.

TIPOS DE EXCEPCIONES



Q Ministerio de Educación (Uso educativo nc)

De forma general, para incluir el manejo de excepciones en un programa hay que realizar los siguientes pasos:

- Dentro de un bloque `try` inserte el flujo normal de las instrucciones.
- Estudie los errores que pueden producirse durante la ejecución de las instrucciones y compruebe que cada uno de ellos provoca una excepción.
- Capture y gestione las excepciones en bloques `catch`.

Sección "try"

Cuando el programador ejecuta un código que puede provocar una excepción (por ejemplo: una lectura o escritura de un fichero), debe incluir este fragmento de código dentro de un bloque `try`:

```
try {  
    ...  
    // Código susceptible de provocar errores  
    ...  
}
```

Sección "catch"

Pero lo importante es cómo controlar qué hacer con la posible excepción que se genera. Para ello se utilizan las cláusulas `catch` que permiten especificar la acción a realizar cuando se produce una determinada excepción.

```
try {  
    ...  
    // Código susceptible de provocar errores  
    ...  
} catch(Excepcion_tipo excepcion) {  
    ...  
    // Código que gestiona la excepción capturada  
    ...  
} catch(Excepcion_tipo_generico excepcion) {  
    ...  
    // Código que gestiona la excepción capturada  
    ...  
}
```

Como puedes ver en el ejemplo anterior se pueden anidar sentencias `catch` para ir gestionando, de forma diferente, diferentes errores. Es conveniente hacerlo indicando en último lugar las excepciones más generales, es decir, aquellas más genéricas (zona superior en el árbol de herencia de excepciones) ya que el intérprete `Java` ejecuta el bloque de código `catch` cuyo parámetro es el tipo de una excepción lanzada.

Sección "finally"

Si desea realizar una acción común a todas las excepciones se utiliza la sentencia `finally`. Este código se ejecuta siempre, con independencia de que haya sucedido una excepción (y se haya tratado en una cláusula

```
catch).
```

```
try {  
    ...  
    // Código susceptible de provocar errores  
    ...  
} catch(Excepcion_tipo excepcion) {  
    ...  
    // Código que gestiona la excepción capturada  
    ...  
} finally {  
    ...  
    // Código que se ejecuta siempre, con independencia de que se  
    // haya producido la excepción  
    ...  
}
```



Para saber más

En el siguiente enlace tienes más información, a nivel teórico, sobre las excepciones:

[Manejo de excepciones \(Wikipedia\)](#) 

Respecto al lenguaje de programación Java dispones de este vídeo que contiene una presentación con un repaso exhaustivo sobre las excepciones:

<https://www.youtube.com/embed/FRz0aVW0msE> (<https://www.youtube.com/embed/FRz0aVW0msE>)

 [Ministerio de Educación](#) (
Licencia estándar de YouTube)

 [Descripción textual del vídeo](#) 

Una vez visionado el material anterior ahora tienes un proyecto de ejemplo con código fuente donde se producen y gestionan diferentes tipos de excepciones:

 [Proyecto Java en NetBeans de ejemplo sobre programación haciendo uso de las excepciones \(PSP06_CONT_R007_ExceptionEjemplo.zip\)](#) (4.8 KB)

Por último, y no menos importante, en la documentación oficial de Java dispones de material de ampliación respecto a todo lo que hemos visto:

[Manejo de excepciones \(Oracle\)](#)  

[Excepciones - bloque 'try' \(Oracle\)](#) 

[Excepciones - bloque 'catch' \(Wikipedia\)](#) 



Autoevaluación

Indica la afirmación incorrecta sobre las excepciones:

- ☐ Se pueden utilizar todas las cláusulas `catch` para gestionar todas las excepciones que consideremos necesarias.
- ☐ Las excepciones se tienen que gestionar de lo más específico a lo más general.
- ☐ EL código en `finally` se ejecuta siempre.
- ☐ Todas las afirmaciones son correctas.

No es correcto, repasa de nuevo el apartado.

Incorrecto, repasa los conocimientos del apartado.

No es la respuesta correcta, repasa otra vez el apartado.

Correcto.

Solución

1. Incorrecto (#answer-19_63)
2. Incorrecto (#answer-19_111)
3. Incorrecto (#answer-19_114)
4. Opción correcta (#answer-19_117)

2.1.1.- Ejemplo

Con objeto de familiarizarnos con la gestión de excepciones veremos un ejemplo de código fuente dónde se genera un vector de 100 elementos con valores aleatorios y, posteriormente, son almacenados en el archivo denominado `salida.txt`

Como se puede apreciar, en las primeras líneas de código donde se generan los valores aleatorios, no hay opción a que se pueda producir ningún tipo de excepción. Pero a la hora de guardar los valores en el archivo podría suceder que no fuera posible abrir o acceder al archivo y, por ello, se produjera una excepción de entrada/salida (`IOException`).

Main.java (escribirNumerosAleatorios)

Sin uso de Excepciones

```
import java.io.FileWriter;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Main {

    // estructura lineal que contendrá los números aleatorios
    private static List numeros;

    // tamaño de la estructura lineal que contendrá los números aleatorios
    private static final int SIZE = 100;

    public static void main(String[] args) {
        escribirNumerosAleatorios();
    }

    public static void escribirNumerosAleatorios() {

        // creamos el objeto en memoria donde almacenamos los números aleatorios
        numeros = new ArrayList<>(SIZE);

        Random randomGenerator = new Random();

        for (int i = 0; i < SIZE; i++) {

            // creación de valor aleatorio numérico contenido entre 0 y 99
            numeros.add(randomGenerator.nextInt(100)); // y almacenamos en la
            // estructura lineal definida anteriormente para ello
        }
    }
}
```

```

// variable para volcar información
PrintWriter flujoSalida = null; // en un archivo

System.out.println("A partir de esta línea debemos definir el bloque

flujoSalida = // como no se ha definido el 'try-catch' el compilador
               new PrintWriter(// devuelve ERROR en la siguiente línea de código
                               new FileWriter("salidaNumerosAleatorios.txt"));

for (int i = 0; i < SIZE; i++) {
    flujoSalida.println("Valor de: " + i + " = " + numeros.get(i));
}

flujoSalida.close();

}
}

```

Otro posible fallo

Al ejecutar la siguiente sentencia dentro del segundo bucle 'for' podríamos tener un problema:

```
flujoSalida.println("Valor de: " + i + " = " + numeros.get(i));
```

Esto es debido a que el valor contenido en la posición solicitada (i) esté fuera del rango de la estructura lineal de datos (en este caso oscila entre las posiciones 0 y 99). Si se tratase de referenciar un elemento fuera de ese rango se generaría la excepción `IndexOutOfBoundsException`.

En el código mostrado en la siguiente pestaña vamos a tratar las dos excepciones comentadas para permitir que la lógica siempre sea ejecutada de forma segura.

Con uso de Excepciones

```

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Main {

    // estructura lineal que contendrá los números aleatorios

```

```

private static List numeros;

// tamaño de la estructura lineal que contendrá los números aleatorios
private static final int SIZE = 100;

public static void main(String[] args) {
    escribirNumerosAleatorios();
}

public static void escribirNumerosAleatorios() {

    // creamos el objeto en memoria donde almacenamos los números aleatorios
    numeros = new ArrayList<>(SIZE);

    Random randomGenerator = new Random();

    for (int i = 0; i < SIZE; i++) {

        // creación de valor aleatorio numérico contenido entre 0 y 99
        numeros.add(randomGenerator.nextInt(100)); // y almacenamos en la
        // estructura lineal definida anteriormente para ello
    }

    // variable para volcar información
    PrintWriter flujoSalida = null; // en un archivo

    System.out.println("A partir de esta línea debemos definir el bloque

    try {
        flujoSalida = // la siguiente línea de código se
            new PrintWriter( // asocia con la excepción 'IOException':
                new FileWriter("salidaNumerosAleatorios.txt"));

        // la siguiente línea de código se asocia con la excepción 'ArrayIndexOutOfBoundsException'
        for (int i = 0; i < SIZE; i++) { // 'ArrayIndexOutOfBoundsException'
            flujoSalida.println("Valor de: " + i + " = " + numeros.get(i))
        }

    } catch (ArrayIndexOutOfBoundsException e) {
        System.err.println("Excepción 'ArrayIndexOutOfBoundsException': " +
            e.getMessage());
    } catch (IOException e) {
        System.err.println("Excepción 'IOException': " + e.getMessage());
    } finally {
        if (flujoSalida != null) {
            System.out.println("Cerrando el fichero");
            flujoSalida.close();
        } else {
            System.out.println("NO se ha abierto el fichero");
        }
    }
}

```

```
}  
    }  
}
```

Descarga del código fuente del proyecto

Una vez visionado el material anterior ahora puedes descargarlo como archivo de proyecto para el IDE NetBeans:

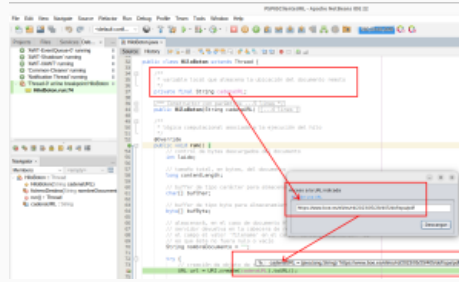


[Proyecto Java en NetBeans de ejemplo sobre programación haciendo uso de excepciones](#)

(3.8 KB)

2.2.- Validación de entradas

La información que introducen los usuarios constituyen una preocupación respecto a la seguridad. Además son una de las principales fuentes de [inconsistencia](#) de datos dentro de un aplicativo.



Q Ministerio de Educación (Uso educativo no)

Un problema de seguridad habitual está asociado con errores relacionados con desbordamiento de unidades temporales de almacenamiento o procesamiento (*buffer overflow*) los cuales se producen cuando se excede el tamaño de una determinada variable o estructura de datos y se logra acceder a posiciones de memoria reservadas. Un buen ejemplo de ello lo tenemos al reservar memoria para un dato como el nombre de usuario (imaginemos que definimos para este uso un tamaño de 20 caracteres) y, de alguna forma, el usuario logra que la aplicación almacene más caracteres. En este caso se está produciendo un *buffer overflow* ya que los primeros 20 caracteres se almacenan en un lugar correcto pero los restantes ocuparán zonas de memoria destinadas a otros fines.

La validación de datos permite:

- Mantener la consistencia de los datos. Por ejemplo al recoger y procesar el DNI del usuario (debe tener siempre el mismo formato).
- Evitar desbordamientos de memoria (*buffer overflow*). Comprobar el formato y longitud de un campo evita muchos problemas asociados a los desbordamientos de memoria.

Para llevar un control riguroso sobre los datos de entrada de los usuarios hay que tener en cuenta la validación del formato y la validación del tamaño de la entrada.

Java incorpora una librería potente y útil, `java.util.regex`, que haciendo uso de la clase `Pattern` nos permite definir expresiones regulares. Con éstas es posible definir exactamente el formato de la entrada de datos.

Para utilizarlas debemos realizar los siguientes pasos:

1. Importación de librería

```
import java.util.regex.*;
```

2. Definición de Pattern y Matcher

```
Pattern verificar = null;  
Matcher matcher = null;
```

3. Compilación del patrón a utilizar

```
verificar = Pattern.compile(patron);
```

El patrón a comprobar es la parte más importante ya que es donde tenemos que indicar el formato que va a tener la entrada.



Para saber más

En los siguientes enlaces puedes encontrar más información sobre la clase `Pattern`

 [Documentación oficial](#) (`Pattern`) 

[Desarrollar y probar Expresiones Regulares](#) 



Autoevaluación

Indica el patrón que permite validar un DNI con la letra en minúscula:

- ☐ `Pattern.compile("[0-9]{9}-[a-zA-Z]");`
- ☐ `Pattern.compile("[0-9][a-zA-Z]");`
- ☐ `Pattern.compile("[0-9]{8}-[a-zA-Z]");`
- ☐ `Pattern.compile("[0-9]{8}-[a-z]");`

No es así.

Incorrecto.

No es correcto, repasa los conocimientos del apartado.

Correcto.

Solución

1. Incorrecto (#answer-23_63)
2. Incorrecto (#answer-23_122)
3. Incorrecto (#answer-23_125)
4. Opción correcta (#answer-23_128)

2.2.1. Patrones clase Pattern

Vamos a ver, de forma resumida, cómo especificar patrones adecuados para la clase `Pattern`:

Elementos para la validación de entradas

Elemento	Comentario.
x	El carácter x.
[abc]	Los caracteres a, b o c.
[a-z]	Una letra en minúscula.
[A-Z]	Una letra en mayúscula.
[a-zA-Z]	Una letra en minúscula o mayúscula.
[0-9]	Un número comprendido entre el 0 y el 9.
[a-zA-Z0-9]	Una letra en minúscula, mayúscula o un número.

Operadores para la validación de entradas

Operador	Comentario
[a-z]{2}	Hay que introducir 2 letras en minúsculas.
[a-z]{2,5}	Hay que introducir de 2 a 5 letras en minúsculas.
[a-z]{2,}	Hay que introducir más de 2 letras en minúsculas.
hola adios	Es la operación OR lógica y permite indicar que se introduzca el texto "hola" o "adios".
XY	Es la operación AND lógica y permite indicar que se deben introducir dos expresiones X seguida de Y.

Operador	Comentario
e(n l) campo	Los delimitadores () permite hacer expresiones más complejas. En el ejemplo, el usuario debe introducir el texto "en campo" o "el campo".

A modo de ejemplo se muestran varios ejemplos de expresiones:

Teléfono

Formato 000-0000000

```
verificar = Pattern.compile("[0-9]{3}-[0-9]{6}");
```

DNI

```
verificar = Pattern.compile("[0-9]{8}-[a-zA-Z]");
```

Provincias andaluzas

```
verificar = Pattern.compile("Almería","Granada","Jaén","Málaga","Sevilla","Cá
```

Ahora vamos a ver cómo aplicar el código anterior en el código fuente de nuestro proyecto:

Paso 1

Le pasamos al evaluador de expresiones el texto a comprobar:

```
comprobar = verificar.matcher(texto_a_comprobar);
```

Paso 2

Comprobamos si hay alguna coincidencia:

```
if(comprobar.find()){  
    // Sí coincide con el patrón  
} else {  
    // NO coincide con el patrón  
}
```


Descarga del código fuente del proyecto

Una vez visionado el material anterior ahora puedes descargarlo como archivo de proyecto para el IDE NetBeans:



[Proyecto Java en NetBeans de ejemplo sobre programación haciendo uso de expresiones regulares para determinar la validez de un DNI](#)

(3.8 KB)

2.3.- Archivos de registro

Los archivos de registro o *logs* permiten almacenar información sobre las distintas acciones que realiza la aplicación o los sucesos que acontecen en el transcurso de éstas. Estos archivos permiten realizar un seguimiento detallado de lo que ocurre en el sistema.



Q Ministerio de Educación (Uso educativo nc)

Java incorpora la librería `java.util.logging` que permite utilizar la clase `logger` para llevar un control de todos los eventos que ocurren a lo largo de la ejecución de la aplicación pero debido a las exigencias de mercado, flexibilidad en el uso y potencia en este caso nos vamos a centrar en otra herramienta, que con similar objetivo, es más popular, versátil y robusta y, por ello, domina el mercado. Se denomina LOG4J (versión 2) y se suele anotar como log4j2. Ha sido desarrollada y es mantenida por The Apache Software Foundation y es un software que se distribuye bajo licencia Apache Software License (version 2.0).

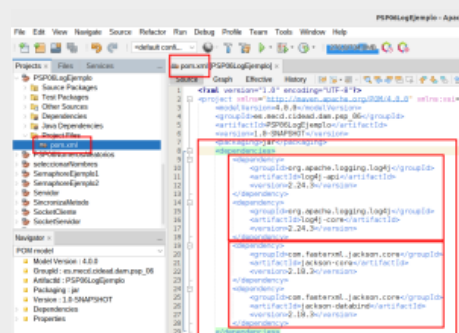
Para desarrollar software con esta herramienta debemos realizar los siguientes pasos:

Gestión de dependencias en el proyecto

Seleccionamos la versión más reciente que contiene los últimos cambios en términos de rendimiento, funcionalidad y seguridad. Las dependencias las gestionamos con [Maven](#), realizando anotaciones en el archivo `pom.xml` poniendo especial atención en las que resultan imprescindibles para tener a nuestra disposición la librería (`org.apache.logging.log4j`) de aquellas adicionales que permiten reconocer configuraciones en formato [JSON](#) el cual no está soportado de forma nativa (en este caso nos decantamos por las librerías de `com.fasterxml.jackson.core`). Entonces, según las indicaciones anteriores, agregamos las siguientes líneas en `pom.xml`:

```
<dependencies>
<dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-api</artifactId>
<version>2.24.3</version>
</dependency>
<dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-core</artifactId>
<version>2.24.3</version>
</dependency>
<dependency>
<groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-core</artifactId>
<version>2.18.3</version>
</dependency>
<dependency>
<groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-databind</artifactId>
<version>2.18.3</version>
</dependency>
</dependencies>
```

Q Ministerio de Educación (Uso educativo nc)



Q Ministerio de Educación (Uso educativo nc)

Puedes descargar el extracto de dependencias del archivo `pom.xml` en

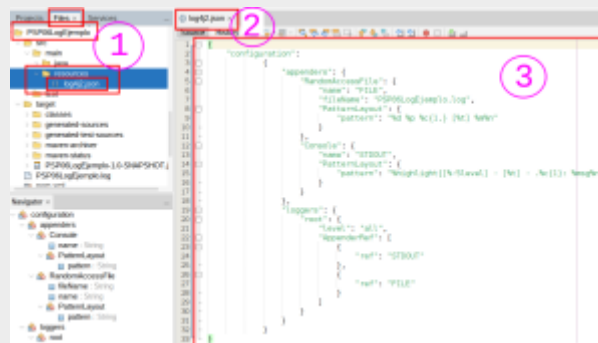
el siguiente enlace:

 [Extracto de dependencias del archivo pom.xml](#) (3.8 KB)

Creación de archivo de configuración de la herramienta Log4j2

Ahora es necesario crear un archivo de configuración (también conocido como archivo de 'properties') para definir el comportamiento que deseamos tenga la integración de logs de la librería log4j2 en el proyecto. Debes de realizar los siguientes pasos:

1. Crea un directorio en el proyecto, seleccionando la vista de archivos al hacer clic en la pestaña 'Files', denominado 'resources'. Deberá estar al mismo nivel que el directorio 'java' (hijo del directorio 'main').
2. Dentro de este directorio recién creado, 'resources', haciendo clic derecho sobre éste, crea un nuevo archivo con el nombre 'log4j2.json' para lo cual deberás especificar 'New' -> 'Other' -> 'Other' -> 'JSON File'.
3. Anota el código de configuración propuesto donde es interesante notar que el archivo '.log' generado se puede definir y, en este caso, lo hemos denominado 'PSP06LogEjemplo.log' y hemos activado la escritura en él, además de consola, con la entrada `"ref": "FILE"` en el nodo `"AppenderRef"`.



Q Ministerio de Educación (Uso educativo nc)

Definición del archivo de configuración de la herramienta Log4j2

Como se indicó en el paso anterior es posible definir un archivo para sobrescribir el comportamiento que por defecto tiene la herramienta log4j2. A continuación se expone el empleado en el proyecto de ejemplo:

```
{
  "configuration": {
    {
      "appenders": {
```

```

        "RandomAccessFile": {
            "name": "FILE",
            "fileName": "PSP06LogEjemplo.log",
            "PatternLayout": {
                "pattern": "%d %p %c{1.} [%t] %m%n"
            }
        },
        "Console": {
            "name": "STDOUT",
            "PatternLayout": {
                "pattern": "%highlight{[%-5level] - [%t] - .%c{1.} %m%n}"
            }
        }
    },
    "loggers": {
        "root": {
            "level": "all",
            "AppenderRef": [
                {
                    "ref": "STDOUT"
                },
                {
                    "ref": "FILE"
                }
            ]
        }
    }
}

```

Puedes descargar el ejemplo de contenido de archivo de configuración en el siguiente enlace:

 [Ejemplo de contenido de archivo de configuración para log4j2](#) (1.1 KB)

Importación de Log4j2 en los archivos .java del proyecto

Importamos las librerías:

```

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

```

Uso/aplicación de *log* en los archivos .java del proyecto

Hacemos uso de los métodos:

```

public class Main {

    private static final Logger LOGGER = LogManager.getLogger(Main.class);

    public static void main(String[] args) {
        LOGGER
            .trace("Anotación Log de nivel 'trace' (traza)");
        LOGGER
            .debug("Anotación Log de nivel 'debug' (depuración)");
        LOGGER
            .info("Anotación Log de nivel 'info' (información)");
        LOGGER
            .warn("Anotación Log de nivel 'warn' (aviso)");
        LOGGER
            .error("Anotación Log de nivel 'error' (error)");
        LOGGER
            .fatal("Anotación Log de nivel 'fatal' (error fatal)\n "
                + "- - - - -");
    }
}

```

Niveles de seguridad de los registros

Los niveles en los *logs* se utilizan para controlar la cantidad y el tipo de información que se añade a las sentencias de registro. Se trata de disponer de un mecanismo que, de forma cómoda, permita ampliar o reducir los avisos o mensajes. El escenario más habitual es aquel en el cual se incrementa la cantidad de información a medida que se trata de averiguar o solucionar un determinado problema detectado en el código fuente. A continuación puedes ver los niveles, definición y significado que hay en log4j2:

Nivel	Importancia	
OFF	0 (Máxima)	Tiene el rango más alto posible
FATAL	100	Eventos de error muy graves q aplicación
ERROR	200	Eventos de error que aún podri
WARN	300	Situaciones potencialmente da
INFO	400	Mensajes informativos que res gruesa

Nivel	Importancia	
DEBUG	500	Eventos informativos detallados
TRACE	600	Eventos informativos más detallados
ALL	Integer.MAX_VALUE (Mínima)	Tiene el rango más bajo posible

Descarga del código fuente del proyecto

A continuación tienes el material con el cual se han realizado las explicaciones y capturas de pantallas anteriores. Puedes descargarlo como archivo de proyecto para el IDE NetBeans:

 [Proyecto Java en NetBeans de ejemplo sobre aplicación y uso del sistema de log log4j2](#)

(4 KB)



Para saber más

En el siguiente enlace puedes ampliar información sobre todos los aspectos relacionados con el funcionamiento y configuración extendida de log4j2:

[Documentación oficial \(log4j2\)](#) 

Respecto a Apache (organización y licencia) te recomendamos acceder a los enlaces que respectivamente amplían detalles sobre ello:

[Web oficial de la organización 'The Apache Foundation'](#) 

[Licencia 'Apache License' \(versión 2\)](#) 



Autoevaluación

¿Qué nivel de registro es el más alto?

- ☐ WARNING
- ☐ DANGEROUS

☐ ALL

☐ INFO

No es correcto, repasa de nuevo.

Incorrecto, repasa los conocimientos del apartado.

Correcto. Tiene el rango más bajo posible, en términos de importancia, aunque el valor numérico es el mayor (elemento por el cual se pregunta y que justifica esta respuesta como correcta). Está destinado a activar todos los registros con lo cual la cantidad de información reportada es la mayor disponible.

No es correcto, repasa el apartado.

Solución

1. Incorrecto (#answer-28_63)
2. Incorrecto (#answer-28_135)
3. Opción correcta (#answer-28_138)
4. Incorrecto (#answer-28_141)

2.3.1.- Ejemplo

A continuación vamos a retomar el proyecto de ejemplo de la sección 2.1.1 en el cual se generan valores aleatorios y se almacenan en un archivo. En este caso se introducen comentarios mediante log4j2 en las diversas secciones (información y excepciones).

Main.java (escribirNumerosAleatorios gestionando excepciones y con integración de [logs](#) )

Extracto del código fuente

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class Main {

    ...

    // definición de variable para incluir sentencias que generen información
    // el log del proyecto
    private static final Logger LOGGER = LogManager.getLogger(Main.class);

    ....

    public static void escribirNumerosAleatorios() {

        ....

        LOGGER.info("A partir de esta línea comienza el bloque 'try'");
        try {
            flujoSalida =                // la siguiente línea de código se
                new PrintWriter(        // asocia con la excepción 'IOException'
                    new FileWriter("salidaNumerosAleatorios.txt"));

            LOGGER.info("Se volcará los números aleatorios generados en el ar
                + " 'salidaNumerosAleatorios.txt'");

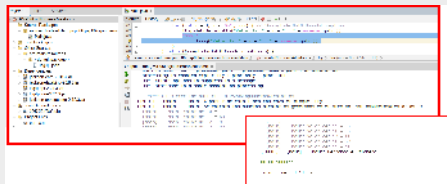
            // la siguiente línea de código se asocia con la excepción
            for (int i = 0; i < SIZE; i++) { // 'ArrayIndexOutOfBoundsException'
                flujoSalida.println("Valor de: " + i + " = " + numeros.get(i))
                LOGGER.debug("Valor de: " + i + " = " + numeros.get(i));
            }
        }
    }
}
```

```

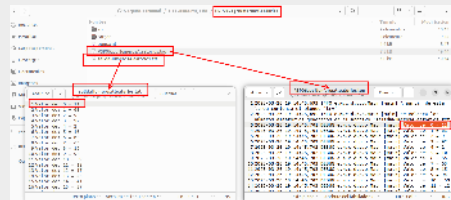
    } catch (ArrayIndexOutOfBoundsException e) {
        LOGGER.error("Excepción 'ArrayIndexOutOfBoundsException': " +
            e.getMessage());
    } catch (IOException e) {
        LOGGER.error("Excepción 'IOException': " + e.getMessage());
    } finally {
        if (flujoSalida != null) {
            LOGGER.info("Cerrando el fichero");
            flujoSalida.close();
        } else {
            LOGGER.info("NO se ha abierto el fichero");
        }
    }
}
}
}

```

Ejecución en consola



[Ministerio de Educación.](#) (Uso educativo nc)



[Ministerio de Educación.](#) (Uso educativo nc)

Descarga del código fuente del proyecto

Una vez visionado el material anterior ahora puedes descargarlo como archivo de proyecto para el IDE NetBeans:

 [Proyecto Java, desarrollado en NetBeans, que integra logs haciendo uso de 'log4j2', sobre un proyecto de ejemplo que calcula números aleatorios y los almacena en un archivo controlando las posibles excepciones](#)

(4.7 KB)

3.- Políticas de seguridad



Caso práctico

Juan: Hola María. Ya he hecho todo lo que me has indicado y por lo tanto mi programa es seguro.

María: Sí, tu programa ha mejorado sustancialmente en términos de seguridad. ¡Buen trabajo! Pero, ¿sabes qué? Todavía es posible hacer algunos cambios más.



🔍 [Ministerio de Educación \(Uso educativo no\)](#)

Juan: ¿A qué tipo de cambios te refieres?

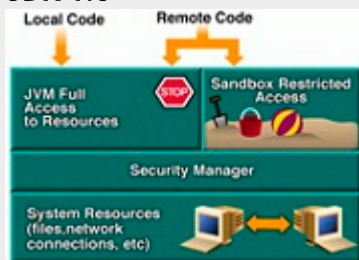
María: Es muy interesante y además, con las políticas de acceso que incorpora Java, resultará sencillo. Con éstas podrás indicar las acciones que puede realizar una determina aplicación en base a su ubicación o firma digital. Es mejor que lo veamos para que valores tú mismo su utilidad.

3.1.- Modelo de seguridad de java

La plataforma Java permite un gran conjunto de características de seguridad: basado en políticas de acceso, fácilmente configurable, control de acceso, servicio criptográfico y gestión de claves, etc.

En la siguiente unidad nos centraremos en la utilización del servicio criptográfico. Por ahora, en ésta, abordaremos las políticas de seguridad de las aplicaciones.

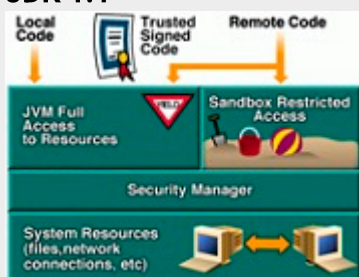
JDK 1.0



Q [JDK 1.0 Security Model](#) (Oracle ©)

Inicialmente, el modelo de seguridad de Java en su primera versión del [JDK](#) era un modelo muy básico ya que las aplicaciones locales tenían total acceso a todos los recursos del sistema y las aplicaciones remotas no tenían acceso total al sistema.

JDK 1.1

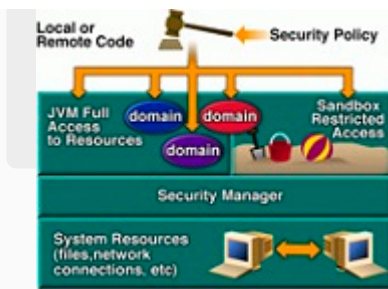


Q [JDK 1.1 Security Model](#) (Oracle ©)

En la siguiente versión, se permitió que las aplicaciones firmadas digitalmente puedan acceder a todos los recursos del sistema.

JDK 1.2

Se incrementa la seguridad del sistema permitiendo establecer las políticas de acceso tanto a las aplicaciones locales o remotas. Una vez que la aplicación es autorizada, se envían las peticiones al Security Manager para poder acceder a los recursos del sistema.



Q [JDK 1.2 Security Model](#) (Oracle ©)

Las políticas de seguridad se pueden establecer utilizando los siguientes elementos:

- Origen (usuario o ruta de acceso de la aplicación).
- Permisos (por ejemplo, se puede indicar si tiene permisos de lectura/escritura sobre un fichero, si puede enviar o no información).
- Destino. Destino al que afecta el permiso. Por ejemplo si trabajamos con ficheros el destino es su ubicación.
- Acción. Las acciones que se pueden realizar sobre el fichero. Por ejemplo, en un fichero los permisos son lectura o escritura.

Aunque existen muchos tipos de permisos, que se pueden especificar en las políticas de seguridad, ahora mostramos aquellos más importantes:

PropertyPermission

Permite establecer los permisos de acceso sobre información del sistema (por ejemplo: sistema operativo, versión de java, directorio home del usuario actual)

[Documentación oficial](#) 

SocketPermission

Permite establecer los permisos sobre las comunicaciones de red. Los permisos se pueden establecer sobre una determinada dirección IP o URL y/o el puerto de comunicaciones al que puede acceder.

[Documentación oficial](#) 

FilePermission

Permite establecer los permisos de acceso al disco de una aplicación. Los permisos se pueden establecer a un determinado fichero o carpeta y, permite establecer los permisos de lectura o escritura.

[Documentación oficial](#) 



Para saber más

Documentación relacionada sobre aspectos de seguridad en Java:

 [Documentación oficial de 'SecurityManager' \(Java SE 23 es la última versión que dispone de esta característica\)](#) 

 ['SecurityManager' deja de estar disponible en Java SE 24](#) 

3.1.1. Modelo de seguridad de Java (2)

Ahora vamos a explorar, en las principales versiones, los cambios más significativos respecto a las mejoras en términos de seguridad en las diversas áreas:

Mostrar

Java SE 8 (marzo 2014)

Cifrado

Listas de control de acceso

Java SE 11 (septiembre 2018)

Cifrado

Java SE 14 (marzo 2020)

Listas de control de acceso

Certificados

Criptografía

Herramientas internas

Java SE 17 (septiembre 2021)

Arquitectura

Java SE 20 (marzo 2023)

Cifrado

Java SE 24 (marzo 2025)

Arquitectura

Seguridad en Java

Introducción

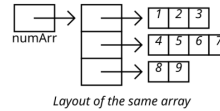
Abordar la seguridad implica prestar atención a los siguientes elementos:

- Conocimiento elevado sobre las características de la infraestructura donde se proporciona/despliega el servicio
- Atención con la gestión de roles (administrador / usuarios)
- Revisar vulnerabilidades (*exploits*): se pueden minimizar (incluso evitar) aplicando programación "de calidad"
- Invertir el tiempo necesario para tener un conocimiento exhaustivo del funcionamiento (lógica y herramientas del lenguaje)
- Implementación de políticas (test, casos de prueba y entornos)
- Cuidado con los errores más frecuentes (evitar "meter la pata")

Desbordamientos

Más en detalle ponemos ahora la atención en los "desbordamientos"

- Inicio y final del tratamiento en estructuras de datos (especial cuidado con las estructuras lineales)

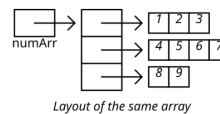


- Hay lenguajes que no evitan problemas (falta de "protección")
- Es necesario conocer las "malas prácticas" para evitarlas
- Las opciones más robustas, en término de lenguajes de programación, no siempre están disponibles o son económicamente viables (tiempo/esfuerzo)

BufferOverflow (I)

Más en detalle ponemos ahora la atención en los "desbordamientos"

- Inicio y final del tratamiento en estructuras de datos (especial cuidado con las estructuras lineales)



- Hay lenguajes que no evitan problemas (falta de "protección")
- Es necesario conocer las "malas prácticas" para evitarlas
- Las opciones más robustas, en término de lenguajes de programación, no siempre están disponibles o son económicamente viables (tiempo/esfuerzo)

BufferOverflow (II)

Las vulnerabilidades de desbordamiento del buffer se producen en todo tipo de software, desde sistemas operativos hasta aplicaciones cliente / servidor y software de escritorio. Esto sucede a menudo debido a una mala programación y la falta de validación de entrada o deficiente en el lado de la aplicación.

Un buffer de memoria es un área en la memoria de la computadora (RAM) destinada a almacenar datos temporalmente. Este tipo de buffers se puede encontrar en todos los programas y se utilizan para almacenar datos para entrada, salida y procesamiento.

BufferOverflow (III)

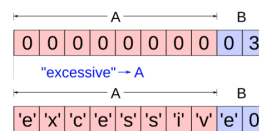
Cuando la cantidad de datos escritos en el buffer excede la cantidad esperada de datos el buffer de memoria se desborda.

Ejemplo: se espera un nombre de usuario que ocupe un máximo de 8 bytes y se proporciona un valor que ocupa 9 bytes el cual es almacenado en la dirección de inicio del buffer.

El exceso de tamaño, 1 byte, provoca un desbordamiento si no es evitado. Los

motivos pueden ser una mala

programación o falta de *desinfección* de entrada.



La explotación de esta vulnerabilidad permite escribir datos maliciosos en memoria y el atacante podría tomar el control del flujo de ejecución de un programa planteando una vulnerabilidad de seguridad muy grave.

BufferOverflow: defensa

En ciertos lenguajes de programación *no seguros* (C y C++ son buenos ejemplos) es responsabilidad del programador evitar estas situaciones ya que el lenguaje no provee de forma nativa protección contra estas situaciones. Es necesario prestar especial atención frente a las siguientes funciones:

Lenguaje de programación C:

```
strcpy()      strcat()
sprintf()     scanf()      sscanf()
gets()        getchar()    fgetc()      getc()
read()
```



Lenguaje de programación C++:

```
vsprintf()    vscanf()      vsscanf()    vfscanf()
```

Es buena idea evitar estas funciones (normalmente hay alternativas que sean potencialmente menos vulnerables)

Seguridad en Java - JVM

La máquina virtual de Java (JVM) implementa la seguridad en base a tres niveles:

- **Cargador de clases** (*class loader*): determina cómo y cuándo pueden cargar código los programas y garantiza que los componentes del sistema no han sido reemplazados
- **Verificador** (*class file verifier*): garantiza que el código tiene el formato correcto, el *bytecode* no viola las restricciones de seguridad y sus instrucciones están asociadas a parámetros de tipo correctos y las pilas internas no podrán desbordarse.
- **Gestor de seguridad** (*security manager*): entre otras funciones controla acceso a los recursos en tiempo de ejecución: E/S de red, archivos, creación de cargadores de clases, ejecución de software externo (perteneciente al SO), carga de código nativo y detención de la JVM.

Seguridad en Java - Security Manager

De cara a la configuración de la herramienta SecurityManager existen cuatro elementos a tener en cuenta

- Identity: bases para las decisiones de seguridad
- Origin: procedencia del código
- Signature: a qué código pertenece (java.security.CodeSource)
- Permission: abstrae el acceso a una operación particular. Incluye un objetivo y una acción. Ejemplos:

```
SocketPermission("https://www.mecd.es/cidead/aulavirtual/login/index.php", "connect")
```

```
FilePermission("/var/www/html/index.html", "**")
```

Seguridad en Java – JCA

Desde el punto de vista de la seguridad el conjunto de clases disponible en las últimas versiones de Java se pueden dividir en dos grupos:

- Clases relacionadas con el control de acceso y la gestión de permisos
- Clases relacionadas con la criptografía

Es precisamente en este último grupo donde localizamos los elementos que forman parte de la **Arquitectura Criptográfica de Java (JCA)**. Los principios de ésta son los siguientes (aunque ahora damos detalles la veremos más adelante en contexto con el resto de elementos):

- Independencia respecto a la implementación
- Interoperabilidad de la implementación
- Extensibilidad algorítmica

A continuación vamos a repasar los elementos principales para entender la gestión de la seguridad en Java.

Seguridad en Java – Elementos (I)

Certification path (ruta de certificación). Se trata de una lista ordenada de certificados, normalmente compuesta por el certificado de clave pública de la entidad final y cero o más certificados adicionales. Una ruta de certificación suele tener una o varias codificaciones, lo que permite su transmisión segura a través de redes y a diferentes arquitecturas de sistemas operativos.

Una ruta de certificación debe validarse antes de que pueda utilizarse para establecer la confianza en la clave pública de un sujeto. La validación puede consistir en diversas comprobaciones de los certificados contenidos en la ruta de certificación, como la verificación de las firmas y la comprobación de que cada certificado no ha sido revocado. Las normas PKIX definen un algoritmo para validar las rutas de certificación compuestas por certificados X.509.

Seguridad en Java – Elementos (II)

JAAS: Java Authentication and Authorization Service (servicio de autenticación y autorización de Java). Se trata de un mecanismo para la autenticación de usuarios, para determinar de forma fiable y segura quién está ejecutando en un momento determinado el código Java.

La autenticación JAAS se realiza de forma *conectable*: permite que las aplicaciones sigan siendo independientes de las tecnologías de autenticación subyacentes. Las tecnologías de autenticación nuevas o actualizadas pueden conectarse a una aplicación sin necesidad de modificar la propia aplicación. Las aplicaciones habilitan el proceso de autenticación instanciando un objeto `LoginContext`, que a su vez hace referencia a un `Configuration` para determinar la tecnología/s de autenticación, o `LoginModule`, que se utilizarán para autenticar. Los `LoginModules` típicos pueden solicitar y verificar un nombre de usuario y contraseña o leer y verificar voz o huella dactilar.

Seguridad en Java – Elementos (III)

Java GSS-API: API Java Generic Security Services (catálogo de servicios genéricos de seguridad de Java). Colección de servicios necesarios para el intercambio seguro de mensajes entre aplicaciones que se comunican.

GSS-API contiene los enlaces de Java para la interfaz de programación de aplicaciones de servicios de seguridad genéricos definidos en **RFC-5653**. Se ofrece a los programadores de aplicaciones un acceso uniforme a los servicios de seguridad sobre una variedad de mecanismos de seguridad subyacentes, incluido **Kerberos**.

Seguridad en Java – Elementos (IV)

JCA: Java Cryptography Architecture (arquitectura criptográfica de Java).

Es una pieza fundamental de la plataforma y contiene una arquitectura de *proveedor* y un conjunto de API's entre las cuales cabe destacar para firmas digitales, mensajes *resumen* (*hashes*), certificados y validación de certificados, cifrado (cifrado simétrico/asimétrico por bloques/por flujos), generación y gestión de claves y generación segura de números aleatorios.

Importante: antes de JDK 1.4, JCE (Java Cryptography Extension) era un producto desagregado y, como tal, JCA y JCE se consideraban componentes separados y distintos. Ahora que JCE está incluido en el JDK, la distinción es cada vez menos evidente. Dado que JCE utiliza la misma arquitectura que JCA, debería considerarse como parte de JCA.

Seguridad en Java – Elementos (V)

JSSE: Java Secure Socket Extension. Permite comunicaciones seguras en Internet. Proporciona un *framework* y una implementación para la versión Java de los protocolos TLS y DTLS e incluye funciones de cifrado de datos, autenticación de servidores, integridad de mensajes y autenticación opcional de clientes.

Con JSSE los desarrolladores pueden garantizar el trasiego seguro de datos entre un cliente y un servidor que ejecute cualquier protocolo de aplicación (como HTTP, Telnet o FTP) a través de TCP/IP.

La API de JSSE soporta los siguientes protocolos de seguridad:

- DTLS: versions 1.0 and 1.2
- TLS: version 1.0, 1.1, 1.2, and 1.3
- SSL (Secure Socket Layer): version 3.0

Seguridad en Java – Elementos (VI)

SASL: Simple Authentication and Security Layer. Es un estándar de Internet ([RFC 2222](#)) que especifica un protocolo para la autenticación y el establecimiento opcional de una capa de seguridad entre aplicaciones cliente y servidor. SASL define cómo deben intercambiarse los datos de autenticación, pero no especifica por sí mismo el contenido de dichos datos. Es un marco en el que pueden encajar mecanismos de autenticación específicos que especifican el contenido y la semántica de los datos de autenticación.

SASL es utilizado por protocolos como LDAP v.3 e IMAP v.4 para permitir la autenticación conectable. En lugar de integrar un método de autenticación en el protocolo, LDAP e IMAP utilizan SASL para realizar la autenticación, permitiendo así la autenticación a través de varios mecanismos SASL.

Credenciales

Toda la información de esta presentación tiene como datos de licencia:

- Autoría: Alejandro Leal Cruz
 - Licencia: Uso educativo-no comercial.
-

3.2.- Firmando ficheros Jar.

Los ficheros JAR (ARchive of Java) contienen componentes de Java (como archivos de clase) y, opcionalmente, recursos de soporte (como imágenes o sonidos).

Como hemos visto antes, las políticas de seguridad se pueden especificar por su ubicación o por la persona que firma el fichero `jar`. A continuación, vamos a aprender a crear un fichero `jar` y firmarlo digitalmente para, más adelante, establecer correctamente sus políticas de acceso.

Los pasos que debemos realizar son:

- Descarga el siguiente código fuente o utiliza cualquier otro código que estimes conveniente.

[Descargar código fuente GetProps.java](#)

NOTA: La consola de comandos deberá ejecutarse como administrador.

- Compila el código fuente:

```
javac GetProps.java
```

- Crea el fichero jar ejecutando:

```
jar cvf GetProps.jar GetProps.class
```

- Genera las claves. A continuación vamos a crear un par de claves (una clave privada y una clave pública) para firmar el fichero. La clave privada la vamos a utilizar para firmar el fichero y la clave pública la van a utilizar los demás usuarios para comprobar que el fichero es correcto. En la siguiente unidad aprenderás muchos más conceptos relacionados con la criptografía en java. Para generar las claves ejecutamos:

```
keytool -genkey -alias firmar -keystore DAM -keyalg rsa -storepass  
distancia
```

donde:

- `-alias firmar`. Indica el alias que se va a utilizar para referirnos al keystore, que es donde se van almacenar las llaves generadas.
- `-keystore DAM`. Indica el nombre del `keystore` que se esta creando o utilizando.
- `-keyalg rsa`. Especifica el algoritmo de encriptación utilizado.
- `-storepass distancia`. Indica el password del `keystore`.

A continuación, tal y como se puedes ver en la siguiente figura, hay que introducir los datos del certificado: nombre, organización, localidad, etc.

```
Símbolo del sistema
D:\syp>keytool -genkey -alias firmar -keystore DAM -keyalg rsa -storepass distancia
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
[Unknown]: ASIR CIPFPD
What is the name of your organizational unit?
[Unknown]: Informática
What is the name of your organization?
[Unknown]: CIPFPD
What is the name of your City or Locality?
[Unknown]: Logroño
What is the name of your State or Province?
[Unknown]: La Rioja
What is the two-letter country code for this unit?
[Unknown]: ES
Is CN=ASIR CIPFPD, OU=Informática, O=CIPFPD, L=Logroño, ST=La Rioja, C=ES correct?
[no]: y
Generating 3,072 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 90 days
for: CN=ASIR CIPFPD, OU=Informática, O=CIPFPD, L=Logroño, ST=La Rioja, C=ES
D:\syp>
```

- Finalmente, firmados el fichero jar utilizando el certificado creado

```
jarsigner -keystore DAM -signedjar sGetProps.jar GetProps.jar
firmar
```

donde:

- `-keystore DAM`. Es el keystore creado anteriormente.
- `-signedjar sGetProps.jar`. Indica el fichero `jar` firmado.
- `GetProps.jar` Es el fichero `jar` a firmar.
- `firmar`. Es el alias que se ha creado anteriormente.

```
Símbolo del sistema
D:\syp>jarsigner -keystore DAM -signedjar sGetProps.jar GetProps.jar firmar
Enter Passphrase for keystore:
jar signed.

Warning:
The signer's certificate is self-signed.
D:\syp>
```

- Exportamos la llave pública del certificado ejecutando:

```
keytool -export -keystore DAM -alias firmar -file PSP.cert
```

```
Símbolo del sistema
D:\syp>keytool -export -keystore DAM -alias firmar -file PSP.cert
Enter keystore password:
Certificate stored in file <PSP.cert>
D:\syp>
```

Ahora ya tenemos todos los archivos necesarios para enviarlos a los diferentes clientes que quieran utilizar nuestra aplicación: el fichero jar firmado (`sGetProps.jar`) y el certificado (`PSP.cert`).



Para saber más

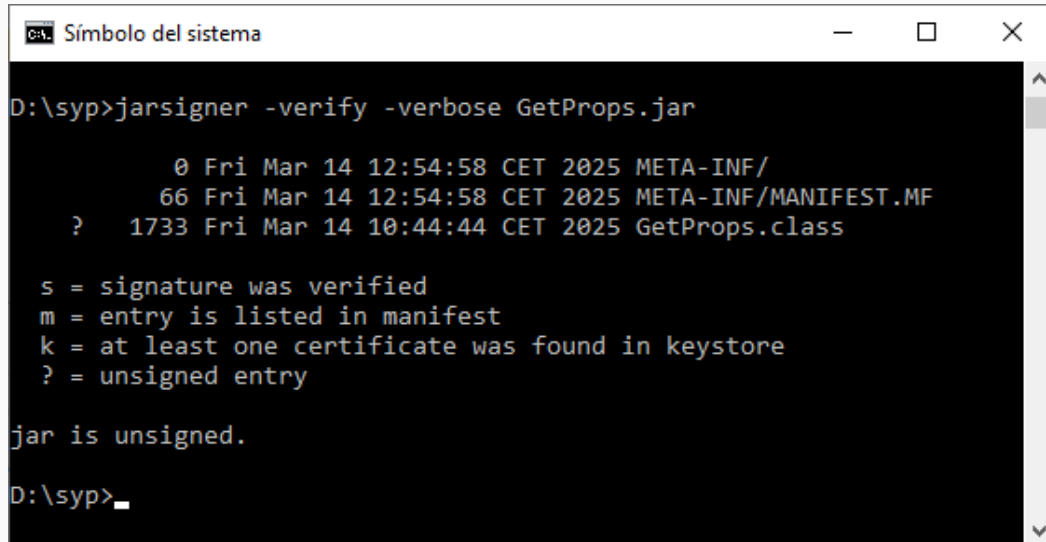
Si deseas conocer el funcionamiento de los certificados de seguridad, consulta el siguiente enlace:

[Certificado digital.](#)

3.3.- Comprobar ficheros Jar firmados.

Una vez que tenemos el fichero jar firmado (`sGetProps.jar`) y el certificado de seguridad (`PSP.cert`), podemos intentar verificar la firma del fichero. Si lo intentamos con el fichero no firmado:

```
jarsigner -verify -verbose GetProps.jar
```



```
D:\syp>jarsigner -verify -verbose GetProps.jar

    0 Fri Mar 14 12:54:58 CET 2025 META-INF/
    66 Fri Mar 14 12:54:58 CET 2025 META-INF/MANIFEST.MF
    ? 1733 Fri Mar 14 10:44:44 CET 2025 GetProps.class

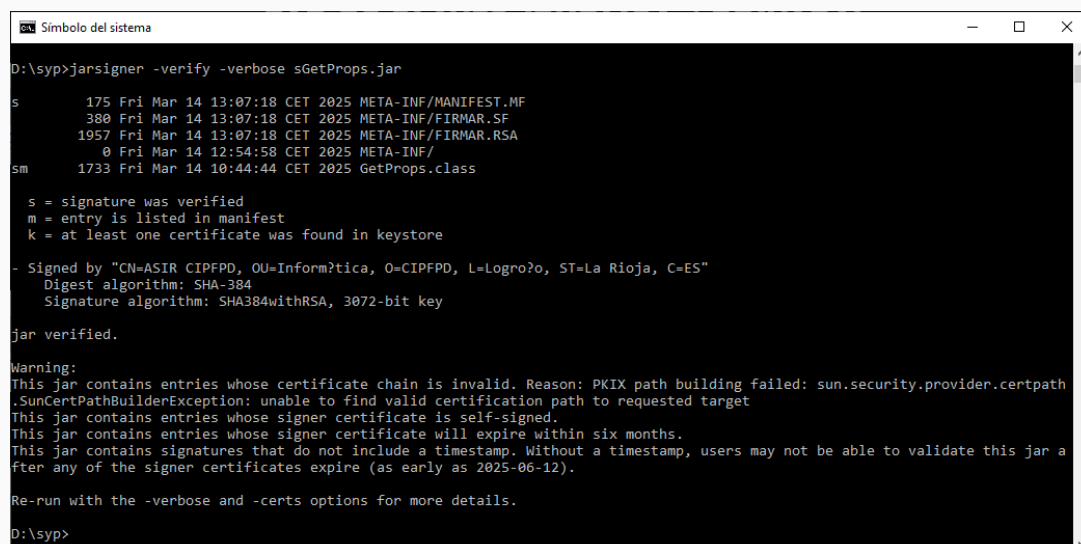
s = signature was verified
m = entry is listed in manifest
k = at least one certificate was found in keystore
? = unsigned entry

jar is unsigned.

D:\syp>_
```

Observamos como nos indica que el archivo no está firmado. Ahora lo intentamos con el fichero firmado:

```
jarsigner -verify -verbose sGetProps.jar
```



```
D:\syp>jarsigner -verify -verbose sGetProps.jar

s      175 Fri Mar 14 13:07:18 CET 2025 META-INF/MANIFEST.MF
      380 Fri Mar 14 13:07:18 CET 2025 META-INF/FIRMAR.SF
      1957 Fri Mar 14 13:07:18 CET 2025 META-INF/FIRMAR.RSA
sm     0 Fri Mar 14 12:54:58 CET 2025 META-INF/
      1733 Fri Mar 14 10:44:44 CET 2025 GetProps.class

s = signature was verified
m = entry is listed in manifest
k = at least one certificate was found in keystore

- Signed by "CN=ASIR CIPFPD, OU=Inform?tica, O=CIPFPD, L=Logro?o, ST=La Rioja, C=ES"
  Digest algorithm: SHA-384
  Signature algorithm: SHA384withRSA, 3072-bit key

jar verified.

Warning:
This jar contains entries whose certificate chain is invalid. Reason: PKIX path building failed: sun.security.provider.certpath
.SunCertPathBuilderException: unable to find valid certification path to requested target
This jar contains entries whose signer certificate is self-signed.
This jar contains entries whose signer certificate will expire within six months.
This jar contains signatures that do not include a timestamp. Without a timestamp, users may not be able to validate this jar a
fter any of the signer certificates expire (as early as 2025-06-12).

Re-run with the -verbose and -certs options for more details.

D:\syp>
```

Aunque obtenemos una serie de avisos, como que el archivo es autofirmado y que la firma caducará en 6 meses, observamos como indica que el archivo está verificado.

Para ejecutar el archivo class firmado, por ejemplo en otro ordenador, deberemos escribir:

```
java -cp sGetProps.jar GetProps
```

```
Símbolo del sistema
c:\syp>java -cp sGetProps.jar GetProps
About to get os.name property value
The name of your operating system is: Windows 11
About to get java.version property value
The version of the JVM you are running is: 23.0.1
About to get user.home property value
Your user home directory is: C:\Users\CIPFPD
About to get java.home property value
Your JRE installation directory is: C:\Program Files\Java\jdk-23
c:\syp>
```



Para saber más

A continuación puedes ver una guía rápida para la firma de aplicaciones.

 [Documentación oficial \(Steps for the Code Signer\).](#)



Autoevaluación

¿Qué comando permite firmar ficheros jar?

- ☐ `policytool.`
- ☐ `jarsigner.`
- ☐ `keytool.`

Incorrecto, repasa los conocimientos del apartado.

Correcto.

No es correcto, repasa los conocimientos del apartado.

Solución

1. Incorrecto (#answer-131_63)
2. Opción correcta (#answer-131_86)
3. Incorrecto (#answer-131_95)

3.4.- Herramientas de seguridad.

El JDK proporciona herramientas que se utilizan para implementar las normas de seguridad de Java. Las herramientas que podemos utilizar son:

- **keytool**. Permite administrar la base de datos de claves y certificados del sistema. Para utilizarla la herramienta ejecuta:

```
keytool [comandos]
```

Si ejecutas **keytool -help** podrás ver la lista de órdenes y sus respectivas opciones.

- **jarsigner**. Permite firmar y verificar la firma de los ficheros jar. Su sintaxis es:

```
jarsigner [opciones] Fichero.jar alias
```

donde

- **Fichero.jar**. Es el fichero **jar** que se firma o verifica
- **Alias**. Es el alias de entrada al almacén de claves que contiene la clave privada que se utiliza para generar la firma. El almacén de claves es una base de datos de claves y certificados que mantiene la herramienta **keytool**.
- Opciones. Son las opciones que utiliza **jarsigner** y que podemos ver al ejecutar simplemente **jarsigner**.



Para saber más

A continuación puedes ver la información oficial sobre las herramientas de seguridad comentadas.

 [Documentación oficial \(keytool\).](#)

 [Documentación oficial \(jarsigner\).](#)

Anexo I.- Tipos de ataques

La siguiente colección muestra, de forma categorizada, los principales tipos de ataques. Es necesario señalar que cada uno de ellos dispone de múltiples variantes y en cada agrupación es posible profundizar para identificar una cantidad mayor lo cual excede el cometido de este documento.

Sistemas

Explotar bugs del software: utilizar fallos de seguridad en el software para atacar un sistema.

Romper contraseñas: fuerza bruta o ataques basados en diccionarios que permiten obtener las contraseñas del sistema o de un determinado servicio.

Red

Barridos de ping: utilización del protocolo ICMP para determinar los equipos activos de una red.

Confianza transitiva: aprovechar la confianza entre usuarios o equipos para tomar sus privilegios.

DNS spoofing: falsificación de una entrada DNS que apunta a un servidor no autorizado

DoS: ataque de denegación de servicio que consiste en saturar un sistema para impedir la utilización correcta del sistema.

Hijacking: permite a un usuario robar una conexión de un usuario que ha sido autenticado en el sistema.

Man in the middle: a través del ataque ARPspoofing el atacante se sitúa en medio de la comunicación entre varios equipos para realizar otros ataques, como sniffer, spoofing, phishing etc.

Mensajes de control de red o enrutamiento fuente: se envían paquetes ICMP para hacer pasar los paquetes por un router comprometido.

Navegación anónima: no se considera directamente un ataque, pero la suelen utilizar los atacantes para realizar sus fechorías. Se denomina "navegación anónima" cuando un usuario utilizar diferentes servidores proxy para ocultar su dirección IP.

Phishing: ataque informático que consiste en falsificar un sitio web para poder obtener las contraseñas de sus usuarios.

Reenvío de paquetes: retransmisión de paquetes para engañar o duplicar un mensaje (p.ej. una transferencia).

Sniffer: programa o equipo que registra todo el tráfico de una red. Se

utiliza especialmente para obtener las contraseñas de los sistemas.

Spoofing: el atacante envía paquetes con una dirección fuente incorrecta. Las respuestas se envían a la dirección falsa. Pueden usarse para acceder a recursos confiados sin privilegios para realizar tanto ataques DoS directos como indirectos o recursivos.

Servidores web

Inyección SQL: ataque que consiste en modificar las consultas SQL de un servidor web para poder realizar consultas SQL maliciosas.

LFI: ataque informático que consiste en hacer que un servidor ejecute un script que está alojado en el mismo servidor.

RFI: ataque informático que consiste en hacer que un servidor ejecute un script que está alojado en una máquina remota.

XSS: consiste en engañar al servidor web para que ejecute un script malicioso en el navegador del cliente que visita una determinada página.

Aplicaciones

Crack: permite romper la protección de una aplicación comercial.

Keylogger: software o hardware que registra todas las pulsaciones de teclado que se realizan en el sistema.

Rootkit: software que se instala en un sistema y oculta toda la actividad de un usuario (el atacante).

Troyano: software que se instala en el ordenador atacado que permite al atacante hacerse con el control de la máquina.

Virus: software que se copia automáticamente y que tiene por objeto alterar el normal funcionamiento del equipo sin el permiso o el conocimiento del usuario.

Varios

Ingeniería social: consiste en convencer a un usuario legítimo para que facilite información (contraseñas, configuraciones, etc.) haciendo uso de manipulación en opiniones externas o mediante requerimientos no legítimos.

Rubber-hosse: hacer uso de sobornos, chantaje o tortura física/mental para obtener una determinada información.